

8. RadixAttention and Prefix Reuse

Chapter Abstract. RadixAttention turns repeated token prefixes into reusable runtime state. The reuse is legal only when logical prefix identity matches live physical KV ownership. When a second request shares the first 64 tokens with a protected cache node, the runtime may skip prefill only if those KV slots are still the right slots for the same prefix.

This chapter starts with prefix identity, radix-tree geometry, and page-aligned reuse. It then follows one prefix hit through matching, request-row projection, and suffix execution. Publication and locks protect live entries. Eviction and observability expose lifetime. Measurement reports reuse without confusing logical prefixes with physical KV.

8.1. The Prefix-Reuse Problem

Two requests can begin with exactly the same token sequence and still fail to reuse work: the earlier request’s KV pages may already have been evicted, moved, or released. Prefix reuse is therefore a runtime proof, not a string comparison. A matched token prefix must still name protected KV state; that distinction separates textual sameness from executable reuse.

Reuse as a state-management problem.

Prefix reuse is a state-management problem built around exact string-prefix matches. In an online server, many requests begin with the same system prompt, tool preamble, retrieval prefix, or conversation history. Recomputing attention over that prefix wastes dense model work and increases time to first token. Reuse is correct only when two facts hold: the new request names the same logical prefix, and the physical KV slots holding that prefix are still alive when attention reads them. The easy mistake is to stop at token equality. A prefix can match perfectly while the KV pages that would make reuse executable have been evicted, moved, or protected by another owner.

Prefix reuse is legal only when identity and KV lifetime agree.

The name RadixAttention is therefore a useful chapter title but a potentially misleading implementation label. The prefix data structure owns identity and matching. It also owns insertion, eviction, and lifetime protection. The attention layer consumes a prefix-aware forward-batch view whose reuse decision has already been made. Prefix matching starts from the Chapter 2 prefix-hit idea (Definition 2.4.1). This chapter first refines that idea into prefix identity (Definition 8.2.1) and radix prefix hits (Definition 8.2.4). It then defines page-aligned reusable prefixes (Definition 8.2.6) and KV-index reuse (Definition 8.2.7). Those decisions happen before the attention backend sees q , k , v , positions, and page metadata [41]. The organizing question is how the runtime turns textual overlap into that executable attention view without confusing logical identity with physical KV state.

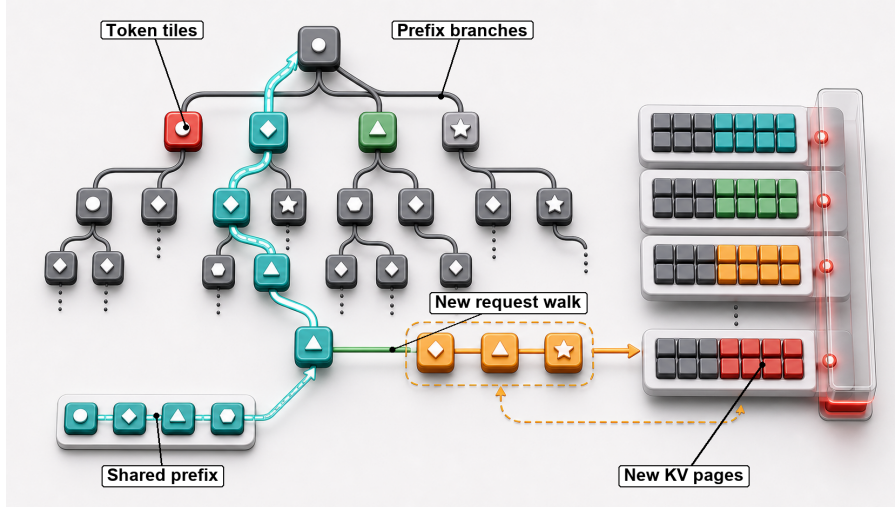


Figure 8.1: Radix tree prefix matching. A radix tree built from token tiles, with a new request walking existing prefix branches before extending into new KV-cache pages.

Figure 8.1 shows that split as a tree walk followed by executable suffix work.

A running prefix example.

The running example returns to the pancake and waffle prompt pair from the runtime preliminaries. The Chapter 1 prompts share five visible prompt pieces; here that shared prefix is compressed to three local token labels so the radix tree remains readable. The shared prompt pieces from the `google/gemma-4-E4B-it` chat service are remapped to:

$$A = r_{\text{pan}} = [11, 12, 13, 91], \quad B = r_{\text{waf}} = [11, 12, 13, 77], \quad C = r_{\text{pan}}^+ = [11, 12, 13, 91, 5].$$

A compact introductory tree assumes page size $P = 1$, stores one shared edge for the common joke-request prefix $[11, 12, 13]$, then branches to the pancake token $[91]$ and the waffle token $[77]$. Request C is a later pancake-continuation request with a prefix hit of four logical tokens from $A = r_{\text{pan}}$, so its effective prompt work is only the uncached suffix $[5]$. When that tail is scheduled for model execution, it is the prefill suffix from Definition 6.2.1. The optimization is useful only if the runtime also preserves the mapping from the four logical positions to the KV-cache indices that store their keys and values.

A prefix hit is a claim about both identity and lifetime.

The lifecycle map.

Prefix reuse is easier to read as a lifecycle than as a catalog of cache classes. Table 8.1 follows the six transitions a request experiences: match identity, project request state, execute suffix work, publish committed state, protect or evict pages, and observe whether a reported hit was executable reuse or only textual overlap.

The pancake/waffle trace needs a compact prefix ledger: which tokens identify the hit, which KV indices are projected, which suffix still runs, and which protected owner keeps the hit safe. Table 8.2 names the symbols that the remaining sections actually use. In the lookup-key row, $I(r, L_r)$ means "the identity material for request r through length L_r ," including tokens, positions, namespace, and

Table 8.1.: Reading contract for Chapter 8. The lifecycle stages put the cache, scheduler, allocator, attention layer, and measurement surfaces in the order in which a request experiences prefix reuse.

Lifecycle transition	Reader contract
Match identity	The radix cache proves that token ids, positions, namespace metadata, and resident or loadable KV state name the same prefix.
Project request state	Physical KV indices become the request’s cached-prefix index vector; the remaining prompt tokens become suffix work and token-budget demand.
Execute suffix work	The attention layer consumes a complete prefix-aware forward batch; it does not re-decide cache legality.
Publish committed state	Finished and unfinished requests insert only committed, page-compatible KV state and return duplicate or unpublishable slots to the allocator.
Protect or evict pages	Locks preserve prefixes still owned by a request, session, load, or write; eviction policy ranks only safe candidates.
Observe and measure reuse	Tests, cache events, and counters check that observed cache hits correspond to real executable reuse rather than textual overlap alone.

model/cache metadata; K turns that material into the key used by the prefix tree. The pancake example therefore reads $I(r_{\text{pan}}, 4) = [11, 12, 13, 91]$ plus its namespace and position metadata, and $K(I(r_{\text{pan}}, 4))$ is the executable cache key, rather than only the text.

Symbol	Meaning	Role in the serving account
r	Request identity	The request whose prompt prefix may match reusable cache state.
x_r	Prompt token sequence	The logical token sequence used to compute a prefix-cache key.
$\kappa_r = K(I(r, L_r))$	Prefix-cache lookup key	The namespace, token, position, and configuration identity used for matching.
v, v_r	Prefix-tree or session owner	The generic cached node, and the terminal owner protecting request r 's hit.
h_r^{page}	Executable hit length	The page-aligned prefix length that can be reused for request r .
V_r	Reusable KV-index vector	The physical slots projected from a cache hit into request-local state.
h_r^{prot}	Protected length	The prefix length whose KV slots remain protected while request r runs.
u_r	Suffix work	The prompt tokens still executed after page-aligned reuse.

Table 8.2.: Local notation for RadixAttention and prefix reuse. The symbols separate request identity, cache keys, owners, executable hit length, KV-index projection, protected length, and suffix work.

8.2. Identity, Page Geometry, and Tree Shape

The runtime trace in Section 8.3 depends on one precise question: when are two prefixes the same reusable state? Prefix reuse answers that question by combining logical identity, executable length, and tree shape. These vocabulary pieces stay together here, while publication, variants, and measurement carry their own examples later in the lifecycle.

Identity Objects.

Core logical objects.

Definition 8.2.1 (Prefix identity) *Prefix identity is the cache namespace for a request prefix: the ordered token sequence, its logical positions, and the cache-relevant namespace metadata that decide whether two prefixes may share KV state. Write it as*

$$I(r, h) = (x_{r,0:h}, 0:h, \eta_r),$$

where η_r is the cache namespace metadata that must match for reuse.

Prefix identity is the semantic equivalence class: it states what must match before reuse is even allowed. The prefix key below is the concrete lookup value that carries this identity into the radix tree. For example, if two requests share token span $[11, 12]$ but one has cache namespace $\eta = \alpha$ and the other has $\eta = \beta$, their prefix identities differ and the lookup must miss even though the visible token span matches.

Table 8.3.: Radix node record. SGLang’s `TreeNode` carries the child map, compressed token label, KV-index payload, lock reference, and access metadata that the prefix-cache contract manipulates.

Record field	Book name	Runtime meaning
children	Children map	Map from the first token or page-sized child key to the next radix node. This prevents sibling scans when many suffixes share one prompt.
key	Token-run label $s(v)$	Compressed edge label from the parent to v ; splitting this label exposes a shorter shared prefix.
value	KV page/slot list V_v	Page-aligned physical KV indices stored for the label span. With paged allocation, consecutive page-sized groups identify the protected KV pages.
lock_ref	Lock/reference count	Positive while an active request, transfer, or cache operation depends on the node; eviction may choose only unlocked candidates.
last_access_time	Last access stamp	Policy metadata refreshed by match and insert, then read by LRU-style eviction strategies.

Definition 8.2.2 (Prefix key) *A prefix key is the concrete lookup object built from prefix identity. It is the value compared against radix-tree edges during matching and the value split or inserted when a reusable prefix becomes cache state. Denote the key builder by $K(\cdot)$; a legal lookup key for request r ’s length- h prefix is $\kappa_{r,h} = K(I(r, h))$.*

The key is what the lookup compares; the radix node is where successful matches store tree structure and KV-index payload. Separating the two keeps namespace identity distinct from the compact tree representation.

$I(r, h)$ is the reuse permission;
 $K(I(r, h))$ is the lookup key.

Definition 8.2.3 (Radix node) *A radix node is a compact tree segment whose edge label stores one nonempty token span and whose value records the physical KV indices associated with that span. The path from the root to a node spells a cached prefix. For node v , write $s(v)$ for its edge span and V_v for its stored KV-index vector.*

A compressed radix tree stores maximal one-child runs as one edge label, so branching occurs only where cached sequences diverge. This tree-shape fact is useful later during insertion and node splitting, but the correctness argument rests on the node value: the path spells a prefix and the value names the KV indices for that prefix.

Example 8.2.1 (Radix node split) Suppose an existing edge stores token span $[10, 11, 12, 13]$ and a new key $[10, 11, 99]$ arrives with page size 1. The longest common prefix is $[10, 11]$, so insertion creates an internal node for that shared span and two children: the old tail $[12, 13]$ and the new tail $[99]$. KV-index payloads stay aligned with the spans they justify: the shared node can name only the shared prefix, the old tail remains on the old path, and the new tail becomes publishable only after its suffix KV state exists.

Algorithm 37: Core radix-tree matching. The walk compares compressed edge labels and splits a partially matched edge only when doing so exposes the exact reusable prefix owner.

Input: Root node v_{root} ; page-compatible key tokens

Output: Matched owner node and matched key length

```

1  $v \leftarrow v_{\text{root}}, i \leftarrow 0;$ 
2 while  $i < |\text{tokens}|$  and  $v.\text{children}$  has  $\text{child\_key}(\text{tokens}[i :])$  do
3    $c \leftarrow v.\text{children}[\text{child\_key}(\text{tokens}[i :])];$ 
4    $m \leftarrow \text{lcp}(\text{tokens}[i :], s(c));$ 
5   if  $m = 0$  then
6     break;
7   refresh  $c.\text{last\_access}$ ;
8   if  $m < |s(c)|$  then
9      $s \leftarrow \text{split } c \text{ after } m \text{ tokens, moving the old tail under } s;$ 
10    return  $(s, i + m);$ 
11   $i \leftarrow i + m; v \leftarrow c;$ 
12 return  $(v, i);$ 

```

The foundational tree operation is compressed-prefix matching with one extra responsibility: if a partial edge match exposes a new shared prefix, the tree must split that edge before later publication can attach KV ownership to the right span. Algorithm 37 writes that match/split operation against the record in Table 8.3. The caller has already built a namespace-aware prefix key and chosen the key-view units that the tree stores; publication later rounds reusable KV state to the cache page geometry before attaching it to a node. The helper $\text{child_key}(\text{tokens}[i :])$ means the lookup unit used in the children map for the remaining suffix: usually the first token at offset i , or the first page-sized/key-view unit when the active cache key compares larger units such as page-aligned spans or bigrams.

Algorithm 37 keeps the structural radix invariant local. First, the walk uses the child map to choose the only edge whose key unit can continue the lookup. Then it compares the remaining key with that compressed edge label. Finally, a full edge match advances to the child, while a partial match splits the edge so the newly exposed node spells exactly the reusable prefix. Later sections layer scheduler lookup, cache publication, and eviction on top of this structural operation: insertion attaches only page-compatible KV state to the owner that matching exposes, and eviction may reclaim only unlocked leaf values.

Worst-case shared prompt shape.

If 1,000 requests share the same 2,000-token system prompt, the compressed tree does not store 1,000 copies of that prompt. After the first insertion, the common prompt is one long edge from the root to a protected node, or $2,000/P$ page units when the cache key is page-aligned. With $P = 16$, that is 125 page units. If every request then inserts a distinct suffix, those suffixes branch under the shared prompt node.

The worst lookup for the shared prompt still has to compare the long label, so the comparison cost is $O(2,000)$ token units, or $O(125)$ page-sized units after page alignment. It does not scan 1,000 sibling prompt copies: the child map selects the prompt edge, and later suffix selection is another map lookup at the shared node. This is the point of the compressed radix shape: the cost follows the prefix length and branch points, not the number of requests that happened to reuse the prefix.

Executable Hit Length.

Hit length and KV meaning.

Definition 8.2.4 (Radix prefix hit) *A prefix hit is the reusable cached portion of a request prefix whose token identity, logical positions, namespace metadata, and live KV slots match the new request. For request r , write h_r for the executable hit length and $V_{r,0:h_r}$ for the reusable KV indices that cover logical positions $[0, h_r)$. The owner that returns those indices must also protect them while r runs.*

A prefix hit names the reusable part of the prompt. Effective prompt work is the remaining execution burden after that hit has been subtracted and layout rules have been applied.

Definition 8.2.5 (Radix effective prompt work) *Effective prompt work is the part of the prompt that still needs model execution after subtracting the reusable prefix hit. It refines the Chapter 2 uncached-suffix idea (Definition 2.4.2) with page geometry: for prompt length L_r and executable hit length h_r^{page} , the suffix work is $u_r = L_r - h_r^{\text{page}}$.*

The reusable state behind a prefix hit is ordinary KV-cache state: for an attention layer, it is the key and value tensors already computed for earlier logical positions. Reuse depends on exact token identity and cache metadata agreement. Two prompts share a reusable prefix of length h only if their first h token ids, logical positions, and cache namespace metadata agree; different chat templates, hidden separators, tokenizer versions, LoRA-style adapter ids, or cache salts used as namespace guards can make same-looking text miss.

Figure 8.2 shows the RoPE-specific version of that test. Token equality reaches the reuse gate, but the cached key is executable only when request-local position metadata and model/layer RoPE configuration still agree with the row attention will consume.

A concrete RoPE failure is a shifted-prefix hit. Suppose the token span $[11, 12, 13]$ was computed at positions 0, 1, 2, but a later request wants to reuse those same token IDs after an inserted system marker, at positions 1, 2, 3. The text span matches, but the rotated key vectors were produced with different position angles. Returning the old KV would make attention read state for the wrong relative offsets, so the hit must be rejected or recomputed at the new positions.

Page Geometry and Tree Shape.

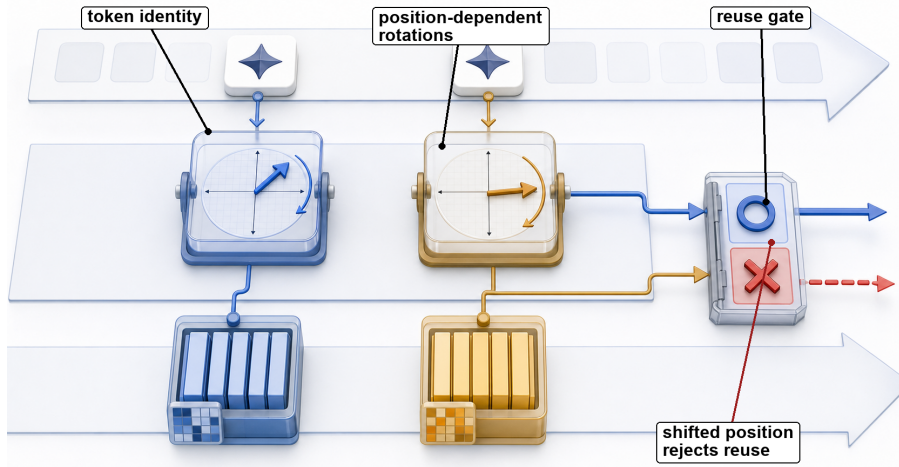


Figure 8.2.: RoPE position agreement as a prefix-cache validity condition. The reuse gate accepts a cached key only when token identity, request-local logical position, RoPE configuration, and layer state all agree; a position shift changes the rotated key even when the token ID matches.

Sharing shape and page geometry.

Definition 8.2.6 (Page-aligned reusable prefix) For page size $P \geq 1$, a logical hit of length h has executable reusable length

$$h^{\text{page}} = P \lfloor h/P \rfloor.$$

The tail $h - h^{\text{page}}$ may remain in request-local KV state while only the page-aligned portion is globally reusable through the radix tree.

h^{page} is globally reusable; the tail can remain request-local.

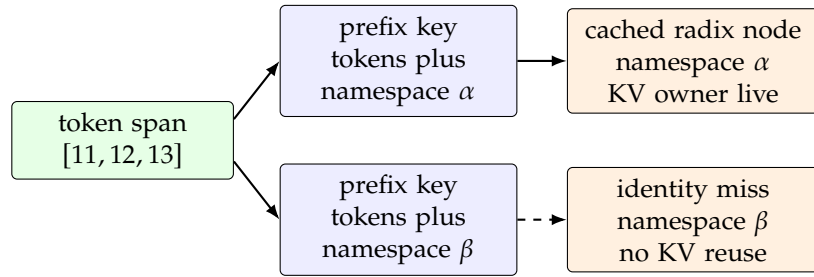
Example 8.2.2 (Prefix hit reduces effective work) For request $C = [11, 12, 13, 91, 5]$, the cached path from request A contributes $[11, 12, 13, 91]$, so $L = 5$, $h = 4$, and the effective prompt work is the one new token $[5]$. If the same textual prefix points to stale KV slots, h must be treated as zero for correctness even though the token strings match.

Example 8.2.3 (Text match without namespace match) Suppose requests U and W both begin with token span $[11, 12, 13]$, but U used adapter namespace α while W uses adapter namespace β . The textual prefix is identical, yet the prefix identities differ. The prefix key for W must miss the cached node for U , so $h = 0$ for that node unless the namespace metadata also agrees. This prevents the radix tree from returning KV slots computed under the wrong runtime conditions. Figure 8.3 shows the same negative case as a cache-key split rather than a prose checklist.

Definition 8.2.7 (KV-index reuse) KV-index reuse means reusing the physical KV-pool index list associated with a matched logical prefix. If a hit of length h returns $V_{0:h}$, then V_i must name the resident KV entry for logical prefix position i ; the tree stores the physical KV indices that point to the per-layer KV tensors.

Prefix identity (Definition 8.2.1), prefix keys (Definition 8.2.2), radix nodes (Definition 8.2.3), prefix hits (Definition 8.2.4), effective prompt work (Definition 8.2.5),

Figure 8.3.: Prefix identity is more than token equality. The same token span can produce different prefix keys; a prefix hit is legal only when namespace metadata and live KV ownership match as well.



and KV-index reuse (Definition 8.2.7) separate the logical and physical sides of reuse. Prefix identity answers whether a hit is semantically legal. The prefix key is the runtime comparison object, the radix node value answers where the already-computed KV entries are stored, and the prefix-hit length determines how much prompt work remains.

The SGLang implementation follows the same split: cache keys carry token identity and namespace metadata, while tree nodes carry physical KV-index tensors and lifetime state. This extends Chapter 7’s request-row and token-slot map: the radix tree decides whether an already-owned slot range can be reused before the scheduler allocates suffix work. Two requests with identical text but different cache namespace metadata must not share a hit, because their physical slots may encode different adapter state, cache versions, or other runtime conditions.

Concept-to-code map

- ▶ **Prefix identity** K_r . Token ids plus namespace metadata map to RadixKey in `python/sglang/srt/mem_cache/radix_cache.py`, including `extra_key`, slicing, page alignment, and prefix matching.
- ▶ **Reusable KV payload**. Radix-tree nodes in the same file carry the matched KV-index tensors, lock state, and policy metadata.
- ▶ **Scheduler lookup key**. `python/sglang/srt/managers/schedule_policy.py` constructs `RadixKey(token_ids=..., extra_key=...)` before prefix-cache lookup and insertion.

Example 8.2.4 (Same-looking text with different tokens) Two clients can send prompts that render as the same visible text while one chat template inserts a different beginning-of-sequence token, role separator, tool delimiter, or hidden system-prompt marker. The radix tree compares token ids and namespace metadata, so the match stops at the last identical token. Prompt hygiene for prefix reuse means stabilizing the tokenizer, chat template, system prompt, tool schema, and cache namespace beyond copying the visible string.

Table 8.4 is a compact checklist for that identity claim. The table is intentionally broader than the ordinary token tree: prefix reuse is valid only when every active identity dimension agrees with the cache path that produced the KV state.

The page size turns this into a systems problem. SGLang rounds cache keys down to a page-aligned length when the configured page size is greater than one, and the prefix matcher returns only the page-aligned portion that is safe to store in the tree. A textual match of 103 tokens with a 16-token page size is therefore a tree hit of 96 tokens, with seven tail tokens handled as ordinary

Reuse surface	Identity rule
Ordinary token prefix	Token ids, logical positions, page-aligned length, and live device KV indices must agree; the tree can reuse only the page-compatible prefix whose KV slots still store the same logical positions.
LoRA or cache namespace	Namespace metadata, including a LoRA-style adapter id or a cache salt when present, prevents sharing token ids computed under different runtime state.
EAGLE bigram view	Overlapping bigram units for the EAGLE family introduced in Chapter 17, shared boundary token, and value length truncated to the bigram count must agree; the compared units are (x_i, x_{i+1}) , not individual token ids.
Streaming session	Session id, committed length, protected length, and session-held request-pool slot define the hit, because the reusable state may live in a session slot rather than as a normal radix-tree node.
Host or storage-backed hit	Host node, host-hit length, load-back state, and storage medium must agree; a semantic host hit is executable only after the needed pages become resident.
Unified or hybrid cache	Full-attention, SWA, Mamba [14], and branch-specific state tracked by the unified cache components decide whether reuse is valid across specialized reusable objects.

Table 8.4.: Prefix-identity matrix for radix reuse. Each row names an additional dimension that can make two visually identical prefixes unsafe to share.

request-local KV state. The benefit of prefix reuse is measured at the boundary between these two lengths: the scheduler saves work for the cached prefix and still must allocate and execute the uncached suffix.

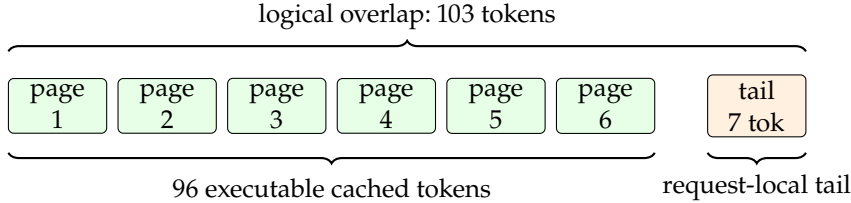


Figure 8.4.: Page-aligned hit and request-local tail. With 16-token pages, a 103-token textual overlap becomes a 96-token reusable hit; the remaining seven tokens may stay in the request row but cannot become global radix-tree state until a later page-compatible transition.

The main correctness boundary is therefore narrow. A match is a promise that the returned indices correspond to the leading logical positions of the same prefix identity. An insertion is a promise that the inserted indices contain KV entries for the logical tokens in the key. An eviction is a promise that only unprotected physical slots are reclaimed. Publication, variants, and sessions reuse this same boundary in later lifecycle sections.

8.3. Match, Project, and Execute Suffix Work

A prefix-aware serving step begins before the model forward and continues after the suffix has executed. This section is the discussion spine: match a legal prefix, project the matched KV indices into request state, allocate and execute only the suffix, then hand the completed context to later publication and eviction transitions. For each candidate request, the scheduler asks the prefix cache for the longest legal hit. The hit length becomes the cached prefix length; the remaining tokens become suffix work. If request r has total prompt length L_r and

page-aligned cached hit length h_r^{page} , then the prefill step accounts for

$$u_r = L_r - h_r^{\text{page}}$$

A prefix hit reduces suffix work before the forward pass begins.

u_r is suffix work after executable page-aligned reuse, not textual matching alone.

new prompt tokens. Prefix matching changes scheduling because it changes the token budget and the number of KV slots the step must allocate. The preceding section formalized the identity and page-geometry terms; the trace here shows how those terms change request state before attention executes.

The savings are prefill-side savings. For prompt length L_r , executable hit length h , and generated length D , prefix reuse removes h prompt tokens from the current request's prompt path and leaves $L_r - h$ prompt tokens to execute. It does not remove the D future decode steps; each generated token still requires a decode iteration over the then-current context. Prefix reuse therefore mainly reduces prefill cost, time to first token, and prefill-side memory allocation.

The match result carries more than a length. It includes the physical address list for the cached prefix, and its terminal ownership object identifies the tree or session state whose lifetime must be protected while the request is active. SGLang updates the request-row table with matched indices and uses the uncached suffix to drive new allocation. In the notation of Definition 7.2.1, that update publishes cached prefix slots into M_r . The attention backend later consumes the complete position-to-slot map, but the split between cached and uncached work is made at scheduling time.

Scheduler-side terms.

Definition 8.3.1 (Prefix-cache contract) *A prefix-cache contract is the runtime interface that maps a legal prefix key to reusable KV indices, a terminal ownership object, insertion semantics, eviction semantics, and lock-reference operations. In compact form, it exposes a partial match map*

$$\mathcal{C}(\kappa) = (h, V, v)$$

from legal key κ to hit length h , KV-index vector V , and terminal owner v , plus insertion, eviction, and locking operations.

The cache exposes that interface to the scheduler. Request prefix state is the scheduler-local projection of one match result into request fields, protected owners, and suffix length.

Definition 8.3.2 (Request prefix state) *Request prefix state is the request-local record that carries a cache hit into scheduling: the matched KV-index vector, terminal owner, optional host-hit metadata, protected length, and resulting suffix length. It is the bridge between the tree's match result and the batch's executable suffix. Write it as*

$$P_r = (V_r, v_r, G_r^{\text{host}}, h_r^{\text{prot}}, u_r),$$

where V_r is the matched device-index vector, v_r the terminal owner, G_r^{host} optional host-hit metadata, h_r^{prot} the protected length, and u_r the remaining suffix.

Case	Match result	Executable suffix	Scheduler consequence
Full hit	$h^{\text{page}} = L_r$ and protected indices cover the prompt.	No prompt suffix.	Admit as already materialized prompt state if decode resources fit.
Partial hit	$0 < h^{\text{page}} < L_r$.	$x_r[h^{\text{page}} : L_r]$.	Allocate and execute only the suffix; protect the hit while the request runs.
Miss	$h^{\text{page}} = 0$.	$x_r[0 : L_r]$.	Treat the whole prompt as ordinary prefill work.

Table 8.5: Prefix matching as a scheduler ledger. The cache result decides which prompt tokens already have executable KV state and which tokens still need prefill allocation and model execution.

Concept-to-code map

- **Prefix-cache operation records.** The lifecycle maps to the `BasePrefixCache` contract and its `MatchPrefixParams`, `MatchResult`, `InsertParams`, and `EvictParams` records.
- **Lookup key.** Candidate token ids and `req.extra_key` map to the `RadixKey` built by scheduling policy before `tree_cache.match_prefix`.
- **Projection tuple P_r .** Returned `device_indices`, terminal node, host node, and host-hit length map to `req.prefix_indices`, `req.last_node`, `req.last_host_node`, and `req.host_hit_length`. Those host fields are forward-compatible metadata for the hierarchical-caching path in the next chapter; the device hit remains the executable state for this chapter's core examples.

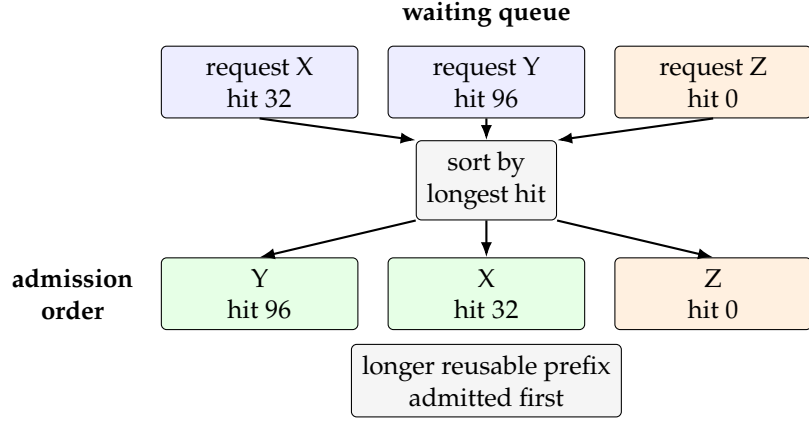
Request preparation then sets `req.cache_protected_len` from the match result when the cache reports one, otherwise from `len(req.prefix_indices)`, and computes `req.extend_input_len` as `len(req.fill_ids) - len(req.prefix_indices)`. Later batch preparation slices `fill_ids[len(prefix_indices):]` as executable input, records prefix and extend lengths, and attributes cached-token counters by device, host, and storage portions when hierarchical hits have been loaded back.

The scheduler projection is the use of that request prefix state (Definition 8.3.2): matched KV indices become the cached-prefix index vector, the protected owner becomes the request's cache lifetime reference, and the executable suffix becomes the token span $x_r[h^{\text{page}} : L_r]$ with length $L_r - h^{\text{page}}$. A cache-aware scheduling policy can then use the same state before the forward pass, for example by prioritizing long existing prefix hits, grouping work by radix-tree locality, or delaying duplicate in-batch shared prefixes until one request populates the cache. The resulting admission account includes cached prefix length, host-loaded portion, executable suffix length, page-alignment overhead, expected output reservation, and the lock that protects the matched prefix while the request runs.

Table 8.5 records the ledger that the following discussion uses.

Figure 8.5 gives a policy-only example: once hits are known, a scheduler may admit longer reusable prefixes before shorter or cold requests.

Figure 8.5.: Cache-aware scheduling by longest prefix match. The scheduler computes prefix hits before prefill admission, so a request with a 96-token reusable prefix can run ahead of shorter or cold candidates while its cache state is still valuable; other policies can use tree locality or priority, but all consume prefix-cache state before the forward step. The request labels here are policy examples, not the pancake/waffle A, B, C trace.



Example 8.3.1 (Scheduler projection from a match result) Suppose the match for the running request C returns physical index vector $[40, 41, 42, 43]$ and $h_C^{\text{page}} = 4$. The scheduler projection writes those four indices into the cached-prefix vector, sets the protected length to four, computes one suffix token, and slices $[5]$ as the only new prefill input. The visual rule is the same as Figure 8.4: cached pages stay at the front of the request row, while the suffix is the part that still needs allocation and model execution. A prefix hit removes prompt-side work before the first generated token; the generated-token loop still runs once per output token, so long outputs can dominate end-to-end latency even with a large hit.

One request through the radix path.

A single prefix hit has to survive several representation changes. At the tree interface it is a key match. In request state it becomes a cached-prefix index vector, protected length, and suffix length. In the batch it becomes a suffix-only input tensor plus output cache locations for newly computed tokens. Inside the attention layer wrapper, it is no longer a tree operation at all; it is a backend call over a forward batch whose position-to-slot map already includes the cached prefix. Table 8.6 keeps those representations aligned for one request.

Example 8.3.2 (Attention consumes a completed context) For the same request $C = r_{\text{pan}}^+$, the attention layer should not inspect the radix tree to decide whether $[11, 12, 13, 91]$ was a legal hit. It receives a forward-batch view whose position-to-slot map already contains $[40, 41, 42, 43]$ for the cached prefix and one newly allocated output slot for token 5. The layer wrapper reshapes q , k , and v , attaches the layer identity and cache-write permission, and delegates to the selected backend. If the prefix legality were re-decided inside the backend, the scheduler’s token budget and the allocator’s KV ownership ledger could diverge.

The introductory tree used page size $P = 1$ to expose the shared $[11, 12, 13]$ edge. In the running scheduler example with page size $P = 4$, request $A = r_{\text{pan}}$ has already inserted the full page $[11, 12, 13, 91]$ with KV indices $[40, 41, 42, 43]$. Request $B = r_{\text{waf}} = [11, 12, 13, 77]$ matches the first three tokens textually, but the

Algorithm 38: Radix prefix lookup for scheduling. A namespace-aware key match becomes executable reuse only after page alignment, physical-index projection, and lock acquisition agree.

Input: Prompt tokens x_r ; namespace metadata η_r ; radix tree $\mathcal{T}_{\text{radix}}$; page size P

Output: Cached prefix indices, suffix work, and a protected terminal owner

- 1 build the namespace-aware key $\kappa_r = K(I(r, L_r))$;
 - 2 walk $\mathcal{T}_{\text{radix}}$ to find the longest prefix-key match for κ_r ;
 - 3 round the executable hit down to the page-aligned boundary required by the KV pool;
 - 4 collect the physical KV indices for the executable hit;
 - 5 increment lock references on the matched path or terminal node;
 - 6 write the cached indices into request row M_r ;
 - 7 set suffix work to the unmatched positions of x_r ;
 - 8 **return** *matched indices, suffix work, and the locked node*;
-

page-aligned hit is zero because the tree stores only complete four-token pages. Request $C = r_{\text{pan}}^+ = [11, 12, 13, 91, 5]$ matches the full first page, so $h_C^{\text{page}} = 4$ and $u_C = 5 - 4 = 1$. The request row for C therefore starts with cached indices $[40, 41, 42, 43]$, allocates one new slot for token 5, and protects the node containing $[11, 12, 13, 91]$ until the step releases it. The arithmetic is small, but it names the radix path’s load-bearing facts: textual overlap, page alignment, physical KV indices, and lock lifetime.

Algorithm 38 turns the radix-cache contract into one reader-checkable listing. First, the scheduler turns request identity into a namespace-aware tree key and finds the longest textual match. Then it rounds that match down to the page geometry that the KV pool can execute and projects the hit to physical KV indices. Finally, it locks the matched owner before writing the request row and scheduling suffix work. The match step is meaningful only because prefix identity, tree geometry, physical projection, and lock lifetime are kept in that order.

Table 8.6.: One request through the radix path. The same prefix hit is represented as a tree match, request-row state, batch metadata, backend-readable context, and finally reusable or evictable cache state.

Trace stage	Runtime state change	Invariant to verify
Match candidate	Build the prefix key from token ids and namespace metadata, then record the matched physical indices, terminal owner, optional host owner, host-hit length, and protected length on the request.	The key namespace, page alignment, and node lifetime must agree with token equality.
Compute suffix work	Set the suffix length to $L_r - h_r^{\text{page}}$ and adjust any output or log-probability offsets relative to the cached prefix.	The scheduler charges only uncached suffix tokens as new prompt work, but the full sequence length still defines attention context.
Prepare extend batch	Slice the suffix token span, record prefix and suffix lengths, and allocate output locations for newly computed suffix KV.	The request row must contain cached indices before the newly allocated suffix locations.
Run attention	Reshape q , k , and v , then delegate to the selected attention backend with the complete forward-batch view.	The backend should see one coherent context; it should not re-decide whether the prefix match was legal.
Publish or continue	Insert committed page-aligned state and free duplicates or unaligned tails; for unfinished requests, rematch the committed prefix and refresh request state for the next chunk.	Only stable, page-compatible KV becomes reusable tree state. Partial tails stay request-local.
Reclaim pressure	Free unlocked leaf values while exposing store and remove events to observers.	Eviction may change capacity and event logs, but it cannot reclaim a node still protected by an active request.

8.4. Publishing Reusable State

A prefix hit becomes durable cache state only after the runtime publishes committed KV indices back into the radix structure. The indices must also be aligned to the page rule. This transition answers a different question from lookup. Lookup asks whether the request may reuse earlier KV. Publication asks which newly computed KV slots are now stable enough to become tree-owned state for a later request. Finished requests, unfinished chunked-prefill requests, continuation caching, duplicate-slot freeing, and unaligned-tail handling are all publication paths, not lookup paths.

Publishing makes KV reusable only after the tree identity and physical slots describe the same prefix.

Definition 8.4.1 (Continuation caching) *Continuation caching is the insertion of committed generated tokens as part of the reusable prefix sequence after a request advances or finishes. In the ordinary SGLang radix path, a finished request inserts committed prompt-token and output-token history up to the committed KV length unless an opt-in mode deliberately keeps only the prompt prefix. If c_r is the committed KV length, the insertion key is built from $(x_r \parallel y_r)[0 : c_r]$ after the active page-alignment rule.*

Concept-to-code map

- ▶ **Continuation insertion key.** The sequence $(x_r \parallel y_r)[0 : c_r]$ maps to `req.origin_input_ids` followed by `req.output_ids`, truncated to the committed KV length and page-aligned before insertion.
- ▶ **Default continuation caching.** Generated continuations are cached by default once they are committed, so later requests can reuse prompt plus generated-token paths.
- ▶ **Prompt-only reasoning mode.** The opt-in `--strip-thinking-cache` server argument skips caching reasoning-model output on finish and keeps only the prompt prefix.

Continuation caching (Definition 8.4.1) makes prefix sharing multi-level rather than one global prompt string. Internal radix nodes may represent a system preamble, tool schema, or few-shot block. They may also represent retrieved context or an earlier turn. Children represent branches such as different questions, options, or reasoning paths. Figure 8.6 shows that tree shape: ordinary prompt components can be shared immediately, while continuation nodes become reusable only after committed generated tokens are published.

Insertion has two common paths. When a request finishes, SGLang inserts the committed prefix represented by the request's input and output tokens. It frees duplicate slots already present in the tree, frees any unaligned tail, and releases the request's previous cache lock. When a request is unfinished, as in the chunked prefill progress invariant (Definition 6.2.5), SGLang inserts the page-aligned committed portion and rematches it. It then writes the new prefix indices back into the request row and keeps any partial-page suffix in request-local state for a later step.

The finished and unfinished paths differ because they have different ownership questions. A finished request is about closing ownership: the tree should keep the reusable prefix, duplicate KV slots should be returned to the allocator,

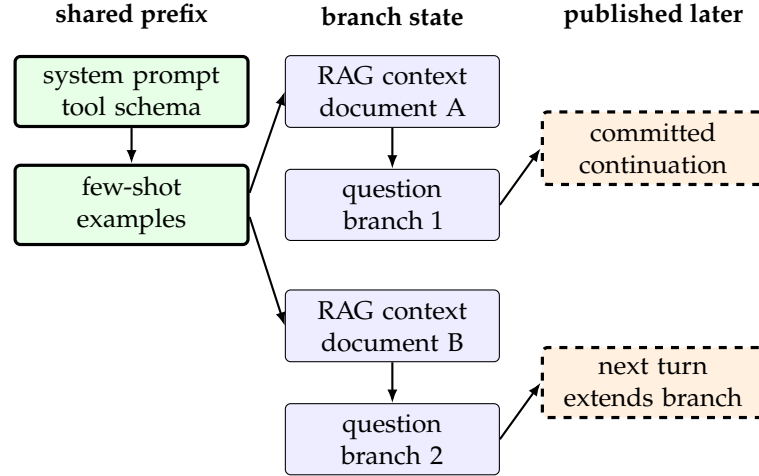


Figure 8.6.: Multi-level prefix sharing and continuation caching. The tree can share nested prompt components; dashed continuation nodes become reusable only after committed generated tokens are inserted as cache state.

and the request should stop protecting the old node. An unfinished request is about advancing ownership: the page-aligned prefix can become reusable now, but the request still needs a coherent row for the next chunk or decode step. The same insertion routine therefore has to update both the global tree and the request-local view.

Table 8.7 summarizes the concrete rows used by the following discussion.

In the running example, request $C = r_{\text{pan}}^+ = [11, 12, 13, 91, 5]$ can reuse the first four positions and execute only token 5. Keep the scheduler’s page size $P = 4$. If C finishes after generating tokens $[6, 7]$, the committed sequence is $[11, 12, 13, 91, 5, 6, 7]$, but the page-compatible committed length is still four. Because the tree already owns $[11, 12, 13, 91]$, publication has no new globally reusable page to insert yet. It should preserve the existing prefix ownership and treat $[5, 6, 7]$ as an unaligned tail: request-local while still needed, or allocator-free once the finished request no longer owns it.

Compaction makes the tree efficient but also raises boundary cases during publication. If a lookup ends inside a stored edge, the radix implementation may split the node to expose a precise match boundary. If two keys diverge after a shared page-aligned prefix, the shared part becomes an internal node and the suffixes become children. The split changes the tree shape while preserving the cached-prefix meaning: the same token positions still point to the same physical KV indices.

Table 8.8 is the concrete version of the split rule. The new request may have computed KV slots $[60, 61, 62]$ for $[10, 11, 99]$, but the tree already owns reusable slots $[50, 51]$ for the shared prefix. The inserted result should publish only the new suffix slot $[62]$ and release duplicate slots for the shared prefix. Insertion is therefore an allocator operation as well as a tree operation.

Algorithm 39 separates cache publication from request continuation. First, the runtime builds the page-aligned reusable key and inserts the corresponding KV slice. The insertion can split compressed radix edges, and the length it returns identifies producer-owned slots that are already represented by existing tree state. Those duplicate slots are safe to release because the tree remains the authoritative owner of the shared prefix.

Publication case	Tree-owned result	Request or allocator result
Finished request	Committed input and output ids become a page-aligned radix key.	The previous prefix lock is released; duplicate or unaligned KV slots are freed.
Unfinished chunk	Only the committed page-aligned prefix is inserted and rematched.	The cached-prefix index vector, terminal owner, and protected length are refreshed for the still-active request.
Continuation caching	Generated tokens become part of the reusable prefix after they are committed.	Later requests that extend the same conversation can hit generated-state KV rather than only prompt KV.
Duplicate prefix slots	Existing tree-owned slots remain authoritative for the shared prefix.	Newly computed duplicate slots are returned to the allocator instead of being kept as a second copy.
Unaligned tail	Partial pages stay outside global radix-tree state.	The tail is either retained as request-local state or freed when the request no longer needs it.

Table 8.7.: Publication paths for reusable radix state. Publication is the point where newly computed KV becomes tree-owned, request-local, or allocator-free state.

Tree object	Before inserting [10, 11, 99]	After the split	KV ownership
Existing edge	[10, 11, 12, 13] $\mapsto$ [50, 51, 52, 53]	[10, 11] $\mapsto$ [50, 51]	keep old shared slots
Old suffix child	absent	[12, 13] $\mapsto$ [52, 53]	remains cached
New suffix child	absent	[99] $\mapsto$ [62]	free duplicate [60, 61]

Table 8.8.: Node splitting preserves prefix meaning while changing tree shape. With page size one, a divergent insertion exposes the common prefix as an internal node and stores only the divergent suffixes below it.

Algorithm 39: Radix insertion after committed work. The tree records only reusable, page-compatible prefix state while request-local state retains any unfinished tail.

Input: Request r ; committed token view K_r ; request KV slots V_r ; previous protected length p_r ; radix tree $\mathcal{T}_{\text{radix}}$

Output: Updated tree ownership, request-row prefix indices, locks, and released KV slots

```

1  $K \leftarrow \text{page\_align}(K_r); V \leftarrow V_r[0 : |K|];$ 
2  $\ell_{\text{dup}} \leftarrow \text{insert}(\mathcal{T}_{\text{radix}}, K, V)$ , splitting compressed edges when a partial match exposes a shared
   boundary;
3 free  $V_r[p_r : \ell_{\text{dup}}]$ , the producer slots already represented by tree-owned prefix state;
4 if  $r$  is finished then
5   | free  $V_r[|K| : ]$ , including any unaligned nonreusable tail;
6   | release  $r$ 's previous prefix lock, if one exists;
7   | return the updated prefix-cache state;
8  $(I, v) \leftarrow \text{match\_prefix}(\mathcal{T}_{\text{radix}}, K)$ ;
9 write  $I[p_r : ]$  into  $r$ 's request row starting at offset  $p_r$ ;
10  $r.\text{cache\_protected\_len} \leftarrow |I|$ ;
11 release  $r$ 's previous prefix lock and acquire a lock on  $v$ ;
12 if  $|I| < |V_r|$  then
13   |  $r.\text{prefix\_indices} \leftarrow I \parallel V_r[|I| : ]$ ;
14 else
15   |  $r.\text{prefix\_indices} \leftarrow I$ ;
16  $r.\text{last\_node} \leftarrow v$ ;
17 return the updated prefix-cache state;
```

Then finished and unfinished requests diverge. A finished request frees the nonreusable tail and releases the old prefix lock. An unfinished request instead rematches the inserted key, rewrites the request row with the authoritative tree-owned indices, advances the protected length, swaps the lock to the new terminal node, and keeps any suffix beyond the rematched prefix in $r.prefix_indices$ for the next chunk or decode step.

The rematch is the non-obvious round trip. The producer may have computed KV slots for the whole committed prefix, but insertion can split a radix edge, deduplicate a prefix that the tree already owns, and free the producer's duplicate slots. After that mutation, the authoritative reusable indices are the tree-owned indices, not necessarily the producer's original slots. An unfinished request therefore reacquires the published prefix from the tree, refreshes its lock, and rewrites its request row before it continues with the remaining tail. This is why publication is followed by lookup-like state repair instead of simply keeping the producer's local vector.

Using Table 8.8, the producer for [10, 11, 99] may arrive with local slots [60, 61, 62]. After insertion, the tree says the shared prefix [10, 11] is already represented by [50, 51], and only the new suffix [99] keeps slot [62]. The request's row must therefore become [50, 51, 62], not [60, 61, 62]; slots 60 and 61 are duplicates and return to the allocator.

8.5. Locks, Eviction, and Memory Pressure

Matching and insertion are only useful while the physical KV slots remain alive. This section isolates the lifetime and pressure side of the radix contract: lock references protect active prefixes, eviction policies rank only safe candidates, and allocator pressure is resolved without breaking logical-prefix ownership.

Locks protect reusable prefixes from eviction while a request still depends on their slots.

Definition 8.5.1 (Paged KV eviction) *Paged KV eviction is memory reclamation over page-compatible cached KV state. The allocator owns physical page capacity, while the radix cache owns the prefix tree that decides which unlocked leaf values may be freed and removed under a configured eviction policy. If $\mathcal{N}_{\text{leaf}}$ is the leaf candidate set and $\text{ref}(v) = 0$ marks an unlocked node, an eviction policy may choose only from $\{v \in \mathcal{N}_{\text{leaf}} : \text{ref}(v) = 0\}$.*

Paged KV eviction is a removal operation. Reference protection is the state in which removal is forbidden because live requests, loads, writes, or sessions still depend on the pages or nodes. A positive reference means an active owner still depends on the node's KV indices, so eviction cannot reclaim that node.

An eviction policy may rank cache nodes by recency, frequency, priority, or insertion order. It may free KV slots only from nodes whose lifetime reference count is zero. Those values must also be free of any active request, write, load, or session owner. A positive lock reference on a terminal node or ancestor protects the corresponding prefix path from reclamation.

Lock references are the lifetime mechanism. SGLang increments lock references along the matched path when a request holds a prefix node, and decrements them when the request releases or replaces that prefix. A node with a positive lock

count is protected from eviction; when the count drops to zero, the node's tokens can move back into the evictable accounting. The lock protects the semantic prefix by protecting the physical slots that implement it.

Example 8.5.1 (Eviction blocked by a live prefix) Assume node $[11, 12, 13, 91] \mapsto [40, 41, 42, 43]$ would be the best eviction candidate under an LRU policy. While request C is using that node as its prefix hit, the node's lock reference is positive, so eviction must skip it and try a different unlocked leaf. Once C releases the prefix, the same node can return to the evictable set. The policy decides priority; the lock decides safety.

Paged KV eviction (Definition 8.5.1) is the inverse transition. The cache chooses evictable leaves according to its eviction priority, frees their KV indices through the allocator, deletes tree leaves, and may move newly childless parents into the eviction candidate set if those parents are not locked. This keeps eviction local to unprotected physical state. It does not invalidate a request that still holds a live prefix reference.

The eviction priority is configurable rather than hard-coded to one recency rule. SGLang's radix cache can rank candidates by recency, frequency, insertion order, reverse insertion order, explicit priority, or segmented recency. The tree nodes therefore carry policy metadata beyond the lock count: access time supports recency choices, hit counts support frequency-sensitive choices, and priority is propagated along inserted paths. The safety rule is independent of the policy. A policy can decide which unlocked leaf is least valuable, but it cannot reclaim a locked node or a page still needed by an active request.

Concept-to-code map

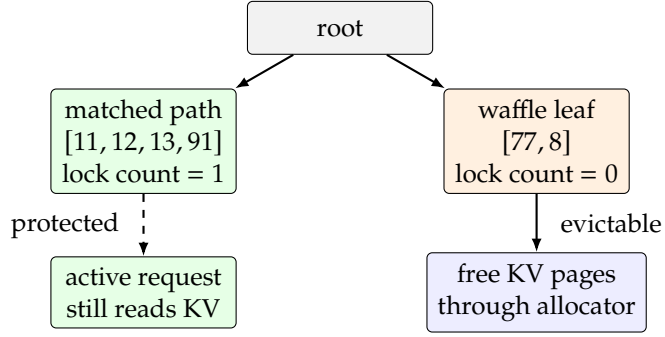
- ▶ **Protected-prefix count.** The lock count λ_v maps to `lock_ref` on radix-tree nodes.
- ▶ **Eviction ranking policy.** Policy choice maps to the `eviction_policy` setting and implementations in `python/sglang/srt/mem_cache/evict_policy.py`; access time, `hit_count`, and propagated priority supply ranking inputs.
- ▶ **Evictable-candidate set.** Cache-side mechanics in `python/sglang/srt/mem_cache/radix_cache.py` map to `RadixCache.evict`, `inc_lock_ref`, `dec_lock_ref`, and `_update_leaf_status`. The policy changes priority order, not the rule that locked nodes stay out of the candidate set.

Figure 8.7 shows the lock rule without the policy details: the active pancake path is protected, while an unrelated waffle continuation leaf can be reclaimed.

Algorithm 40 makes the same concept-to-code map's eviction safety gate explicit. Candidate ranking, lock accounting, and leaf-membership maintenance are separate responsibilities: the policy changes priority order, not the lock rule.

The ledger check is the same one used in the request trace: each transition must update request-visible state and physical KV ownership together. Bugs appear when a scheduler row still names a prefix after the cache has released it, or when an allocator frees a page before the lock reference protecting its prefix has been decremented.

Figure 8.7.: Paged KV eviction with reference protection. Locked radix paths protect the physical KV pages used by active requests; only unlocked leaf values can be freed and removed from the tree.



Algorithm 40: Radix eviction with lock accounting. Eviction policy selects among unlocked leaves, while lock references preserve any prefix still owned by an active request.

Input: Eviction target n_{tok} tokens; radix tree $\mathcal{T}_{\text{radix}}$; KV allocator A ; eviction priority $\text{prio}_{\text{evict}}$

Output: Freed KV slots and updated evictable/protected accounting

```

1   $\text{ref}(v)$  denotes a node's lock count, and  $V(v)$  denotes its KV-index value list;
2  build a priority heap from  $\mathcal{T}_{\text{radix}}$ 's evictable leaves;
3   $\text{freed} \leftarrow 0$ ;
4  while  $\text{freed} < n_{\text{tok}}$  and the heap is nonempty do
5      pop the leaf  $v$  with smallest priority  $\text{prio}_{\text{evict}}(v)$ ;
6      if  $\text{ref}(v) > 0$  then
7          skip  $v$ , because an active request still protects it;
8      else
9          free  $v$ 's KV indices through  $A$ ;
10         remove  $v$  from the radix tree and eviction candidate set;
11         record a KV-cache remove event for  $v$ 's page range;
12          $\text{freed} \leftarrow \text{freed} + |V(v)|$ ;
13         if  $v$ 's parent has no children and parent lock count is zero then
14             insert the parent into the heap with priority  $\text{prio}_{\text{evict}}(\text{parent}(v))$ ;
15 return the number of freed KV slots;
  
```

8.6. The Attention Execution Handoff

The cache can prove a prefix hit, but the model step still needs a complete attention view. This boundary matters because the radix tree, scheduler, and allocator decide legality and ownership before the layer wrapper and backend consume the resulting batch metadata.

The handoff has four roles. The cache proves which prefix is reusable. The scheduler budgets only the suffix that remains. The allocator supplies the KV addresses for the reusable prefix and the newly allocated tail. The backend then executes attention over that assembled view. At the layer call, q , k , v , and layer identity are no longer enough. Cache-write permission and backend metadata also have to agree with the reuse decision that the cache and scheduler have already made.

From prefix hits to executable attention.

The prefix cache sits between the scheduler and the physical KV allocator. The scheduler needs a cached-prefix length so it can budget the suffix. The allocator needs concrete KV indices so it can avoid allocating slots for the part that already exists. The attention backend later sees only executable position and page metadata; it does not decide whether a prefix match was legal.

The prefix cache bridges scheduler budgets and allocator addresses.

The opening distinction between the radix cache and the attention layer wrapper becomes operational at this boundary. In extend mode with a forward context, the wrapper enters SGLang’s unified attention custom operation and lets that operation write through the selected attention backend; otherwise it dispatches directly to the selected backend with the layer object and forward-batch view. The radix tree has already decided which prefix indices are reusable. The attention layer is the execution surface where q, k, v , cache-write permission, layer id, and backend metadata meet.

Concept-to-code map

- **Prefix-cache records.** Shared match, insert, and eviction records map to `python/sglang/srt/mem_cache/base_prefix_cache.py`.
- **Device-resident tree.** Matched radix state maps to `python/sglang/srt/mem_cache/radix_cache.py`; the allocator supplies the page-aware physical KV locations used by cache values.
- **Scheduler projection.** Match results become request state and token budgets in the scheduler path before model execution.
- **Attention execution surface.** `python/sglang/srt/layers/radix_attention.py` consumes the final attention view after prefix reuse and allocation. In the layer wrapper, `RadixAttention.forward` dispatches through `forward_batch.attn_backend.forward` when it does not enter the unified attention operation. Variant-specific cache shapes and streaming-session ownership modes use the same handoff rule.

Contract handoffs.

The prefix-cache boundary is the point where logical reuse becomes a concrete claim on live KV slots. The radix cache compares prefix identity, returns matched KV indices, protects the matched length, and records lifetime facts while the request uses those slots. It does not decide how much free memory exists or how the allocator turns token counts into page ownership. That lower-level capacity problem remains with the KV allocator.

The exported prefix-cache record should therefore be concrete enough to audit: matched length, physical KV indices for the matched span, the protected radix node or virtual-session node, lock or reference-count lifetime, and the suffix length that still needs allocation. A neighboring subsystem may transform those fields into pages, attention metadata, or cache events, but it must not invent a new prefix identity after the match has been accepted.

Prefix reuse exports identity and live indices; allocation owns raw capacity.

Example 8.6.1 (Request C at the execution handoff) For the running request $C = r_{\text{pan}}^+ = [11, 12, 13, 91, 5]$, the accepted hit is the concrete record

$$h_C = 4, \quad V_C = [40, 41, 42, 43], \quad u_C = [5],$$

with the radix node for $[11, 12, 13, 91]$ protected while the step runs. If the allocator gives the suffix token output slot 44, the attention-facing view contains cached prefix slots $[40, 41, 42, 43]$, query position 4, sequence length 5, and output cache location 44. The scheduler budgets one suffix token, the allocator does not allocate the four cached slots again, and the backend reads those slots without re-deciding whether the prefix match was legal.

Hierarchical caching keeps the same distinction after KV pages leave GPU memory. A host or storage hit can prove that a semantic prefix exists outside the device pool; executable attention input requires loading those pages into device-addressable KV memory. The handoff here is the point where “matched prefix” stops being enough.

The attention boundary is similar. Prefix reuse can produce a complete attention view for a step: cached prefix indices, suffix output locations, sequence lengths, and request order. Attention backends may then format page tables, choose kernels, prepare fixed-shape execution metadata, or specialize decode and extend paths. They consume the reuse decision; they do not decide whether two request prefixes were allowed to share KV state.

The same handoff rule applies to nearby runtime features. Radix reuse may interact with chunked prefill, streaming sessions, speculative decoding (Chapter 17), and backend metadata, but each feature has a separate control problem. The cache exports identity, index, protected-length, and lifetime facts; the neighboring runtime path decides capacity, movement, backend execution, speculative acceptance, or user-visible session policy. Table 8.9 summarizes those contract boundaries.

In source, this boundary is the same handoff map as above viewed from the contract side. The prefix-cache parameter and result records, ordinary match and insertion paths, unfinished insertion, lock-reference updates, and eviction preserve the identity, position, page, and lifetime invariants. The match helper, request-round initialization, and extend-preparation path then turn the matched prefix into request-row state and suffix work.

Boundary surface	Prefix-cache contract	Responsibility beyond prefix reuse
KV allocation	Matched prefix lengths and physical KV indices that must remain live while a request uses them.	Free-list policy, request-row allocation, page-pool capacity, and low-level slot ownership mechanics.
Hierarchical caching	The logical prefix identity and radix node that make a host or storage hit meaningful.	Load-back scheduling, lower-tier storage policy, copy completion, and when a nonresident hit becomes device-executable.
Attention backends	A complete attention view: cached prefix indices, suffix output locations, sequence lengths, and request order.	Backend-specific page-table formatting, fixed-shape execution planning, kernel selection, and workspace layout.
Speculative decoding	Committed prefix state and the EAGLE bigram key view used when speculative paths need prefix-cache identity.	Draft proposal generation, target verification, acceptance accounting, and which speculative tokens become committed request state.
Runtime coordination and sessions	Cache ownership facts: protected prefix length, terminal node or virtual session node, and which KV slots can be freed.	Client-visible session semantics, retry policy, abort handling, and cross-request lifecycle orchestration.

Table 8.9.: Contract handoffs around radix prefix reuse. The prefix cache exports identity, index, protected-length, and lifetime facts; adjacent runtime subsystems decide capacity, residency movement, backend execution, speculation, and user-visible lifecycle policy.

8.7. Variants and Streaming Sessions

The core lifecycle is stable across ordinary device-resident full-attention KV spans and specialized cached objects. Variants may change the key view. They may also change the cached state or the owner that protects the state. They should preserve the rule that a hit is legal only when logical identity, executable length, physical state, and lifetime ownership agree.

The same prefix-cache rule extends to cache variants. The reusable state may be unified radix-tree state, SWA state, Mamba state [14], streaming-session state, host-cache hits, or storage-backed hits.

Variant state changes the cache key; a hit is valid only under the same layout contract.

Specialized key views.

Keys can change the unit that the radix tree compares. In the ordinary path, the key iterates over token ids plus namespace metadata. For EAGLE-style speculative matching, whose draft-and-verify algorithm choices are developed in Chapter 17, the same raw token sequence can be reinterpreted as overlapping bigrams.

Concept-to-code map

- **Key unit and namespace.** Ordinary token keys, optional `extra_key`, and bigram-aware length, slicing, value, and match operations map to `RadixKey` in `python/sglang/srt/mem_cache/radix_cache.py`.

- **Speculative bigram consumer.** EAGLE-style speculative decoding consumes prefix-cache matching through representative paths such as `python/sglang/srt/speculative/eagle_info.py` and `python/sglang/srt/speculative/multi_layer_eagle_worker.py`.
- **Cache-aware scheduling key.** `python/sglang/srt/managers/schedule_policy.py` supplies `extra_key` when building keys for cache-aware scheduling.

For example, the raw token span $[a, b, c, d]$ has four ordinary token units but three EAGLE-style bigram units:

$$(a, b), \quad (b, c), \quad (c, d).$$

The slice containing the first two bigram units must still remember the raw boundary token c , because the second unit is (b, c) . The bigram slice therefore keeps one shared boundary token, and the associated value tensor is truncated to the number of logical bigram units. Prefix reuse still requires identity, executable length, and protected state to agree, but the matched unit is an overlapping pair rather than a single token.

Variant cache state.

The unified and Hi/Mamba radix-cache variants extend the same match, lock, and insertion contract to model families whose reusable state can extend beyond a full-attention KV span. The LMCache-backed radix cache shows the same boundary for a storage-backed hierarchy: the key can prove the semantic prefix while residency changes where the physical pages must be loaded from before attention can execute. The registered radix-cache tests cover server-level cache hits, attention-layer integration, sliding-window attention (SWA) radix-cache paths, and unified radix-cache cases; Chapter 7 introduces SWA with KV pool variants.

Example 8.7.1 (Variant state changes what a hit proves) Let page size be $P = 4$, and let request $V = [21, 22, 23, 24, 30]$ match a cached prefix $[21, 22, 23, 24]$. In an ordinary device-resident full-attention path, a hit of four tokens can be projected directly as device KV indices, for example $[80, 81, 82, 83]$, and the suffix work is token $[30]$. In a host-backed path, the same logical key may first identify a host-resident prefix; the scheduler can call it reusable only after load-back makes the four positions device addressable. In a unified or hybrid cache path, the token key must agree with the SWA or Mamba-side state carried by the match and the model branch that will consume the batch. The arithmetic $5 - 4 = 1$ is unchanged, but the reusable object changes from ordinary device KV to loaded-back pages or variant-specific state.

Streaming-session cache state.

Streaming sessions add one more ownership mode to the same lifecycle. In a normal radix hit, the terminal tree node is the object whose lock protects the reused KV slots. In a streaming session, the runtime can keep a request-pool slot alive across turns and restore it into a later request. The prefix hit is therefore not discovered only by walking the tree; it can also be recovered from a

per-session slot that remembers committed length, allocated length, protected length, SWA state, and Mamba state.

Concept-to-code map

- **Session-level prefix owner.** Streaming-session cache state maps to `StreamingSession`, `SessionSlot`, `restore_to_req`, and the streaming `match_prefix` implementation in `python/sglang/srt/session/streaming_session.py`.
- **Restorable request state.** Each session slot stores `req_pool_idx`, committed and allocated KV lengths, cache-protected length, `last_node`, SWA lock metadata, and Mamba pool state.
- **Scheduler wrapper.** `python/sglang/srt/managers/scheduler.py` wraps the tree cache with `StreamingSession` when streaming session mode is enabled.

Example 8.7.2 (Two-turn streaming-session hit) Suppose a session has committed 64 tokens from its first turn, has allocated 72 KV slots, and uses page size $P = 8$. The next turn begins with the same 80-token conversation prefix. Restoring the session slot gives a reusable committed length of

$$\min(64, 80) = 64,$$

so the scheduler treats tokens 64, ..., 79 as the new suffix work. The allocated-but-uncommitted tail from slots 64, ..., 71 is orphaned and can be freed on an eight-token page boundary before the new suffix is allocated: $72 - 64 = 8$, exactly one page for $P = 8$. The returned owner is a virtual session node, so ordinary tree lock-reference updates do not turn the session-held row into an evictable radix-tree leaf. When the session is released, the runtime releases the original tree lock and frees the session-owned request-pool and variant state.

Table 8.10 summarizes the concrete rows used by the following discussion.

Streaming transition	Prefix-reuse consequence
Restore active slot	The request recovers its request-pool row and committed KV length before suffix allocation, so the scheduler can treat the session-held prefix as cached work.
Free orphaned tail	Allocated-but-uncommitted decode or speculative slots above the restored prefix are freed on page boundaries before new extend allocation overwrites the row.
Return virtual node	Ordinary lock-reference updates are no-ops for the session sentinel, so session-held KV does not masquerade as an ordinary radix-tree lock.
Save finished turn	The request-pool row, committed length, allocated length, SWA state, and Mamba state move into the session slot; the request itself is marked as no longer owning that KV.
Release session	The original tree lock is decremented, unprotected session-held KV slots are freed, request-pool slots are returned, and Mamba pool entries are released.

Table 8.10.: Streaming-session prefix reuse. Session slots preserve executable KV state across turns while keeping tree locks, page-aligned tail cleanup, and request ownership explicit.

The session slot may make a later turn look like a prefix hit, but the runtime must still know which KV slots are committed, which tail slots can be freed, which tree lock protects the first turn's inserted prefix, and which state is held

outside the tree’s evictable-size accounting. Streaming-session state belongs in the radix lifecycle because it demonstrates prefix reuse as an ownership contract backed by tree lookup.

8.8. Correctness Invariants and Negative Cases

Prefix reuse is correct only when a logical prefix identity and a physical KV-slot list stay synchronized across every transition. For request r , a valid reuse decision means $V_r[i]$ stores the KV entry for logical position i of $I(r, h_r)$, for every $0 \leq i < h_r$. The terminal owner v_r prevents those entries from being reclaimed while r uses them. The invariants in this section are the compact checks that prevent a text match from becoming a stale KV read, a misplaced batch row, or a misleading cache event.

For a request whose first 64 tokens match a protected cache node v_7 , the reuse state might be $I(r, 64)$ with $h_r = 64$. Its KV-index vector might be $V_r = [420, \dots, 483]$, and its owner might be $v_r = v_7$. Reusing the same V_r after v_7 is evicted would read stale physical state even if the prompt text still matches.

The same mapping must survive the whole lifecycle. Table 8.11 spells out the review points before the formal rule.

The prefix cache is correct when the following invariants hold through match, insert, scheduling, and eviction.

The cache invariants protect both logical prefixes and physical slots.

Invariant 8.8.1 (Prefix-cache alignment). *A prefix-cache reuse decision is valid only when prefix identity, scheduler mode, logical position, page geometry, lifetime, batch row, mutation event, and suffix accounting all describe the same logical prefix and the same physical KV state.*

- **Identity alignment.** A prefix hit is legal only when the token ids and cache namespace in the lookup key match the cached path. Identical text is not enough if the runtime namespace differs.
- **Mode alignment.** Cached prefix state is consumed only as prefix state for a compatible extend or mixed step. A hit returned for reusable prompt KV cannot be treated as newly generated decode KV, and request-local tails that are not publishable under the current cache mode remain outside the tree.

Table 8.11.: Lifecycle points where prefix identity and physical KV state must stay aligned.

Phase	State that must agree	Failure if stale
Match and scheduling	Prefix identity, protected length, and suffix budget.	The scheduler skips prompt work whose KV state is not usable.
Batch construction and attention	Request row, physical KV indices, positions, and output cache locations.	The backend reads or writes rows for the wrong request.
Insertion and events	Published prefix, mutation event, and lifetime owner.	Observability reports a hit before the reusable state is durable.
Session transfer and eviction	Session ownership, lock state, and evictable pages.	A live request keeps an index to reclaimed KV storage.

- **Position alignment.** The i th returned KV index names the KV entry for the i th logical position of the matched prefix. Node splitting may rearrange tree structure, but it must not reorder the value tensor.
- **Page alignment.** When page size is greater than one, tree state is page aligned. Partial pages remain request-local until they can be committed without inventing a reusable prefix that the allocator cannot represent.
- **Lifetime alignment.** Locked nodes and their ancestors are not evicted while a request, write, or load operation depends on them.
- **Batch alignment.** Prefix hits, protected lengths, and request-row updates travel with the same batch order used by the scheduler. Filtering, merging, or reordering a batch cannot attach one request's matched KV indices to another request's suffix work.
- **Event alignment.** Cache mutation events report the same logical tree relationship as the mutation that produced them. A store or remove event must name page-aligned KV data with the parent hash needed by downstream cache observers. Here the parent hash is the stable identifier for the parent prefix-tree node, so an observer can reconstruct where the mutated KV pages sit in the prefix relationship.
- **Suffix accounting.** Cached and uncached tokens are counted in the same logical sequence. A hit of h_r tokens on a prompt of L_r tokens creates suffix work $L_r - h_r$ inside that sequence.

For a concrete batch-alignment failure, suppose request R has prefix indices $[420, \dots, 483]$ for a 64-token hit while request S is a miss whose first suffix token is 22. If a filtering or priority step reorders the batch but leaves R 's prefix vector attached to S 's suffix row, the attention backend receives a coherent tensor shape with the wrong causal history. The fault is not a tree lookup error; it is a request-row alignment error after the lookup.

Example 8.8.1 (Position and lifetime breaks for request C) For $C = [11, 12, 13, 91, 5]$, the valid cached-prefix vector before executing the suffix $[5]$ is $V_C = [40, 41, 42, 43]$: slot 40 stores position 0's token 11, slot 41 stores position 1's token 12, slot 42 stores position 2's token 13, and slot 43 stores position 3's token 91. A position break that swaps the last two entries to $[40, 41, 43, 42]$ keeps the tensor length at four but changes the causal history to $[11, 12, 91, 13]$ for the suffix query at position 4. A lifetime break is just as concrete: if the protected node is unlocked after matching and slot 43 is recycled for the waffle branch before C 's suffix runs, the backend still receives $V_C = [40, 41, 42, 43]$, but the last KV row no longer represents token 91. Both faults can pass shape checks; the invariant rejects them because position and lifetime no longer describe the same prefix state.

These invariants explain why prefix reuse failures are often visible far from the cache code. If identity alignment fails, attention can read KV entries for a different logical prefix. If mode alignment fails, the runtime can publish KV state through the wrong cache path. If position alignment fails, the model sees a causal history with swapped locations. If lifetime alignment fails, the allocator can recycle a slot that a later backend still reads. If batch alignment or suffix accounting fails, the scheduler may under-allocate memory, skip tokens that should have been computed, or bind one request's reusable prefix to another request's row.

Table 8.12 makes the review checks explicit before the prose follows individual failure cases.

Table 8.12.: Failure-to-surface map for the prefix-cache alignment invariant. The table keeps the non-obvious failure surfaces; identity, position, lifetime, and suffix checks are carried in the invariant list and negative examples.

Invariant facet	Example failure	Source or test surface
Page	A partial page is published as globally reusable cache state.	Unfinished-request publication and protected-length checks.
Batch	A filtered or merged batch attaches one request's prefix to another request's suffix.	Extend-batch preparation and attention integration tests.
Event	Store or remove events preserve page bytes but attach the wrong parent hash.	Store/remove event publication and KV-event consumers.

Concept-to-code map

- ▶ **Identity and lookup.** Namespace identity maps to `RadixKey.extra_key`; matched-prefix lookup maps to `RadixCache.match_prefix`.
- ▶ **Publication and lifetime.** Partial-page publication maps to `cache_unfinished_req` and `cache_protected_len`; lifetime protection maps to `inc_lock_ref`, `dec_lock_ref`, and `RadixCache.evict`.
- ▶ **Batch and event alignment.** Batch alignment maps to `ScheduleBatch` extend preparation and `RadixAttention` integration tests; event alignment maps to `_record_store_event`, `_record_remove_event`, and their consumers.
- ▶ **Suffix and bigram accounting.** Executable suffix length maps to `match_prefix_for_req` plus `set_extend_input_len`; EAGLE-style bigram conversion maps to `RadixKey.maybe_to_bigram_view`.

Table 8.13 gives three compact negative tests. Each one is compact enough to look like a harmless cache hit until the missing identity, lifetime, or key-shape condition is named.

Table 8.13.: Negative tests for prefix reuse. A cache hit is valid only after the runtime rejects namespace mismatches, stale physical slots, and key-shape confusion between ordinary tokens and EAGLE-style bigram units.

Invalid reuse case	Why it is invalid	State that should reject or expose it
Same token ids, different cache namespace	The resident KV state was computed under a different logical cache identity, such as a different LoRA id, tokenizer/model assumption, or cache salt.	Namespace metadata and namespace-aware cache-hit tests.
Tree match after KV-slot lifetime ended	The textual prefix still has a tree shape, but a reused physical-index entry names a slot that eviction or request release made available for other state.	Lock count, eviction, allocator ownership, and remove events.
EAGLE bigram length treated as ordinary token length	Four raw tokens form only three overlapping bigram units, so raw-token length and radix-key length disagree by one in bigram mode.	Bigram-key conversion, bigram matching, and page-aligned suffix accounting.

Page alignment is the easiest part to underestimate. A request may have a cached-prefix index vector longer than the page-aligned tree hit because the request row can keep a partial tail that the tree refuses to publish. SGLang tracks this with request-local protected lengths so the unaligned portion can be freed or

committed at the next finished or unfinished cache transition. That distinction preserves both reuse and memory safety.

Unfinished insertion is where the protected-length field matters most. During chunked prefill, SGLang inserts only the page-aligned committed portion, re-matches the inserted key, writes the rematched indices back into the request row starting at the previously protected boundary, and then advances that boundary to the length actually protected by the tree. SGLang also avoids incrementing popularity counters for chunked insertion, so a request does not inflate the popularity of nodes that it created in earlier chunks. A partial page may still live in the request's local indices, but it is not counted as a globally reusable tree prefix until a later transition can publish it safely.

The ordinary radix cache also emits mutation events. Store and remove event publication records per-page mutations, and event draining hands those events to consumers with parent-hash context. That event stream is why event alignment belongs beside position and lifetime alignment: a cache observer that sees the right pages under the wrong parent hash can preserve tensor shape while losing the prefix relationship the radix tree proved.

8.9. Observability and Contract Tests

Prefix reuse should be observable as a sequence of cache-interface facts before it is interpreted as aggregate latency or throughput. The cache event stream, registered tests, benchmark request shape, and per-request hit geometry need to agree on the same prefix relationship.

An observable prefix hit is a reuse decision that can be audited outside the tree implementation. Request metadata and cached-token counters can expose it. Device, host, or storage breakdowns can expose it too. KV-cache mutation events or benchmark traces can provide the surrounding evidence. A useful observation reports the hit length, visible KV-location or tier evidence, and the event trace that justifies the reuse decision.

Contract tests should exercise both the hit path and the miss or eviction path.

Interface records and cache events.

SGLang names the cache interface with parameter and result records, and the ordinary radix cache exposes mutation events through the same observable contract.

Concept-to-code map

- ▶ **Prefix-cache interface records.** `MatchPrefixParams`, `MatchResult`, `InsertParams`, and `EvictParams` map to `python/sglang/srt/mem_cache/base_prefix_cache.py`.
- ▶ **Base radix mutations and events.** `RadixCache.match_prefix`, `insert`, `evict`, page-level store and removal events, page hashes, parent context, and `RadixCache.take_events` map to `python/sglang/srt/mem_cache/radix_cache.py`.
- ▶ **Variant-specific record extensions.** `Hierarchical`, `Mamba`, and sliding-window fields map to `python/sglang/srt/mem_cache/hiradix_cache.py`,

Algorithm 41: Publishing radix cache mutation events. The tree’s match and eviction logic remains local, while page-level stored/removed events expose stable lineage to downstream cache observers.

Input: Cache mutation m ; radix node v ; page size P ; event flags; optional storage medium

Output: Cache-mutation events appended to the publication queue

```

1 if cache events are disabled then
2   | append no events for mutation  $m$ ;
3 if  $m$  inserts, materializes, removes, or evicts  $v$  then
4   | compute  $v$ ’s page hashes if they are not already cached;
5 if  $m$  inserts a new radix node or materializes one in storage then
6   | emit one stored-block event per page with parent lineage, LoRA namespace, and storage
   | medium;
7 if  $m$  removes or evicts a radix node then
8   | emit one removed-block event per page hash and storage medium;
9 if  $m$  clears all cache blocks then
10  | append a clear-all-blocks event;

```

python/sglang/srt/mem_cache/mamba_radix_cache.py, and

python/sglang/srt/mem_cache/swa_radix_cache.py.

► **Observable cache events.** BlockStored, BlockRemoved, AllBlocksCleared, and StorageMedium map to python/sglang/srt/disaggregation/kv_events.py; scheduler metrics consume tree_cache.take_events() through python/sglang/srt/observability/scheduler_metrics_mixin.py.

Algorithm 41 focuses on the publication surface around radix-tree mutations. First, insertion computes any missing page hashes and emits stored-block events with parent lineage. Then removal or eviction reuses the same page-hash view to emit removed-block events for the affected pages. Finally, a clear-all mutation publishes an all-blocks-cleared event, and the separate drain operation atomically returns the queued events to a consumer while replacing the queue with an empty one. The useful abstraction is that each page-sized cache mutation becomes a stable event with enough lineage to let another component follow physical-cache state without replaying the tree operation that produced it.

Example 8.9.1 (Page events for one inserted prefix) With page size $P = 2$, inserting the prefix [11, 12, 13, 91] below parent hash h_0 produces two stored-block events. The first event records page [11, 12], block size two, namespace α , storage medium device, parent hash h_0 , and a new page hash h_1 . The second event records page [13, 91] with parent hash h_1 and page hash h_2 . If pressure later evicts the second page, the removed-block event names h_2 and the same storage medium. A consumer can reconstruct the prefix lineage $h_0 \rightarrow h_1 \rightarrow h_2$; without the parent hashes it would see page mutations but not the radix relationship that made the hit meaningful.

These observability surfaces are useful only when they preserve alignment: variant cache state must name the same logical prefix under its representation, session restore must resume the same request-local state, page events must keep parent-hash lineage, and registered tests must exercise the tree, allocator, and attention handoff from the request surface rather than only the tree implementation.

Test surface	Contract signal
Server cache-hit regression	Repeated multi-turn prompts confirm that logical prefixes become observable server-side cache hits; this checks the request surface together with tree internals.
Attention integration regression	Chunked prefill under FCFS, longest-prefix-match (LPM), and non-overlap LPM exercises the boundary among prefix matching, allocation, and attention execution.
Sliding-window attention regression	SWA-model KL-divergence regression checks show that reuse remains valid for windowed KV state.
Unified-cache regression	Full-attention, Mamba-hybrid, and SWA unified-radix paths cover prefill hits, decode hits, branch interleaving, and model-family-specific state under the unified tree contract.

Table 8.14.: Registered radix-cache tests as contract coverage. The tests make the prefix-cache invariant concrete from the request surface: a textual prefix hit is valid only when the runtime can protect and reuse the corresponding physical cache state through the whole step.

The registered test files behind Table 8.14 include:

- ▶ `test_radix_cache_hit.py`;
- ▶ `test_radix_attention.py`;
- ▶ `test_swa_radix_cache_kl.py`;
- ▶ `test_unified_radix_cache_kl.py`.

Table 8.14 also shows why no single unit test is enough for prefix reuse. Tree operations can be locally correct while a scheduler policy, model-family cache shape, or attention wrapper still breaks the request-level contract. These observability surfaces also define the measurement boundary: first expose the cache facts, then interpret latency or throughput. The measurement section uses those same surfaces as evidence rather than treating aggregate benchmark numbers as a substitute for executable-reuse counters.

8.10. Measurement Evidence and Direct Trace Plan

Prefix reuse needs measurements that separate useful prefix matches from the suffix work and allocator pressure that remain after a hit. Aggregate latency and throughput are downstream effects; the primary evidence is hit geometry, protected-token state, eviction outcome, and realized suffix work. The observability surfaces in Section 8.9 define where those facts appear; the current evidence records direct trace rows for hit geometry, page alignment, and pressure-induced eviction.

This section uses the Chapter 2 Thor/Qwen measurement slice. Table 8.15 is the canonical result surface because it joins request ids with cache events instead of inferring reuse from aggregate serving metrics. Values are device means over joined request rows; raw run identifiers and proxy cache-report rows are retained in the supplementary coverage artifact, `RESULT_COVERAGE.md` in the Volume I Thor experiments directory. The reviewed rows use Qwen 2.5 0.5B Instruct on the Jetson Thor measurement slice. The trace and pressure families include an accepted pressure row using low static-memory allocation with CUDA graph replay disabled; the corresponding graph-enabled pressure setup did not complete a comparable benchmark row.

Reuse claims need hit-length and eviction-pressure measurements.

Table 8.15 summarizes the direct rows. The event/request-id column counts parsed radix-cache event records and the distinct benchmark request ids joined with nonempty radix-event counters; it is not the total request count for the benchmark. Per-device ranges are reported as ranges, not as individual host rows.

Table 8.15.: Direct radix-trace evidence. Hit and suffix columns are device means in tokens per joined request. Events and request-id counts are rounded device means from the parsed trace files. The first four rows are geometry rows; the final row is the accepted no-graph pressure row.

Scenario axis	Events / request ids	Hit	Suffix	Tail/eviction
Short shared prefix	352/70	45.3	150.3	No tail; no eviction
Long shared prefix	357/70	1,560.3	582.0	No tail; no eviction
Intended capacity pressure	1,006/198	354.1	242.4	No eviction in the geometry row
Page size 16	362/70	669.3	428.0	Mean inserted tail 7.3; no eviction
Tighter pressure, no graph	3,107/518	1,001.5	1,047.9	108–138 evictions/device; 82–803k evicted tokens/device

Figure 8.8 plots the same trace geometry as a reviewed-row range plus mean marker. The figure makes the shared-prefix and page-alignment contrasts easier to scan, while Table 8.15 remains the exact reviewed-number record.

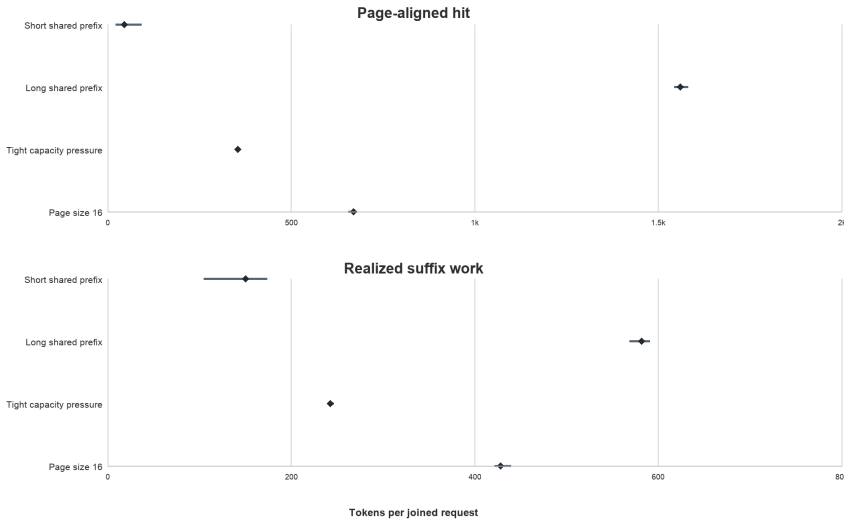


Figure 8.8.: Direct radix-trace geometry for the reviewed Thor/Qwen geometry rows. Diamonds are cross-device means, and horizontal bars show the reviewed range. This plotted run did not trigger eviction; the separate no-graph pressure row in Table 8.15 records the measured eviction outcome.

SGLang already has the workload side of this experiment. The serving benchmark can generate shared-prefix traffic with knobs for group count, prompts per group, system-prompt length, question length, output length, request ordering, and routing-key emission. Those knobs control textual overlap and arrival order, but they do not by themselves prove executable reuse. The direct trace row in Table 8.16 supplies the cache-side counterpart: each benchmark request is joined with the match result, page-aligned hit, protected-token count, and eviction outcome observed at the scheduler/cache boundary.

The remaining limitation is curve coverage, not the existence of eviction evidence. The no-graph pressure row proves that the instrumentation observes eviction

events and evicted tokens under tighter memory pressure, but it is one pressure point rather than a sweep. The row schema below is the direct-measurement shape for pressure curves, recomputation counters, or failed-admission cases.

Field group	Required fields
Hit geometry	logical hit length, page-aligned hit length, unaligned tail length, realized suffix work
Lifetime and pressure	locked token count, evictable token count, protected token count, cache capacity at the decision point
Eviction outcome	evicted token count, eviction policy, recomputed suffix token count, failed-admission cause
Latency and throughput	TTFT, inter-token latency, generated-token throughput, total-token throughput, request E2E latency
Workload controls	shared-prefix group count, prompts per group, system-prefix length, question length, output length, request ordering, routing-key mode
Provenance	model, page size, cache capacity setting, scheduler policy, hardware, SGLang package or commit, run seed or trace id

Table 8.16.: Minimum radix-cache event row for direct measurements. The row separates textual overlap from executable page-aligned reuse and from the memory pressure caused by protecting or evicting cache state.

Table 8.17 makes the counter request actionable. It names where each field is observed so that a benchmark run can distinguish a real executable hit from a prompt-shape coincidence.

Concept-to-code map	
► Workload driver.	Direct prefix-reuse measurements start from <code>python/sglang/bench_serving.py</code> .
► Prefix-hit observation.	Matching and effective hit length map to <code>match_prefix_for_req</code> in <code>python/sglang/srt/managers/schedule_policy.py</code> .
► Executable suffix length.	Request-length setup maps to <code>Req.set_extend_input_len</code> in <code>python/sglang/srt/managers/schedule_batch.py</code> .
► Session reuse and cache events.	Streaming-session hits map to <code>StreamingSession.try_match_prefix</code> ; publication, eviction, event rows, and event draining map to <code>cache_finished_req</code> , <code>cache_unfinished_req</code> , <code>RadixCache.evict</code> , <code>take_events</code> , <code>BlockStored</code> , <code>BlockRemoved</code> , and <code>AllBlocksCleared</code> in <code>python/sglang/srt/mem_cache/radix_cache.py</code> .

Observation point	Fields to join with each benchmark request
Prefix matching	Logical key length, page-aligned device hit length, host-hit length, terminal device owner, terminal host owner, and protected length.
Extend-batch preparation	Executable suffix length, cached-token breakdown, request-row prefix indices, and new allocation demand.
Finished and unfinished publication	Inserted length, duplicate KV slots freed, unaligned tail slots released or retained, and chunked-prefill protected length.
Eviction and eviction helpers	Requested eviction tokens, actually freed tokens, policy choice, protected tokens, evictable tokens, and failed-admission cause when capacity is still insufficient.

Table 8.17.: Instrumentation points for direct radix-cache measurements. The benchmark supplies workload shape; the scheduler, cache, session, allocator, and event surfaces supply executable-reuse evidence.

The clean experiment is a matched workload sweep. Hold model, tokenizer, arrival trace, output lengths, page size, and scheduler policy fixed while changing shared-prefix length, cache capacity, or reuse distance one axis at a time. A positive prefix-reuse result shows lower realized suffix work and fewer allocations for cache hits; a negative or capacity-limited result shows the saved work disappearing through eviction, failed admission, or recomputation counters rather than only through aggregate latency.

8.11. How Prefix Reuse Evolved

Prefix reuse widened in four stages. First, tree-local matching had to preserve compact token paths, value tensors, node splitting, lock references, and eviction over device KV indices. Second, scheduler-visible reuse made a match result part of request state. Cached-prefix indices, terminal owner, protected length, and suffix token budgets had to move together.

Third, host-cache and decode-side paths made the match visible outside the original prefill process. A hit could now depend on page boundaries, namespace metadata, unfinished requests, host-cache events, and decode-side ownership. Fourth, tests and KV event emission made reuse auditable: a trace can distinguish a real executable hit from a textual overlap that never became safe KV state.

The implementation history therefore shows RadixAttention becoming a shared scheduling and memory mechanism rather than a single tree data structure. The attention layer wrapper dispatches backend calls; the radix cache owns matching, locking, insertion, eviction, and reuse accounting.

The evolution trail links cache invariants to ownership and lookup speed.

Concept-to-code map

- ▶ **Shared cache mechanism.** The history of matching, locking, insertion, eviction, and reuse accounting maps to `python/sglang/srt/mem_cache/radix_cache.py`.
- ▶ **Attention-layer execution boundary.** The runtime point where prefix reuse meets backend attention maps to `python/sglang/srt/layers/radix_attention.py`.
- ▶ **Regression surface.** Server-level and integration expectations map to registered radix-cache tests under `test/registered/radix_cache/`.
- ▶ **Request-state and operation records.** Later refinements moved key utilities onto `RadixKey`, exposed `prefix_indices`, `last_node`, `cache_protected_len`, and suffix-length state, and connected the cache path to `MatchPrefixParams`, `MatchResult`, `InsertParams`, and `EvictParams`.

The same contract widened again when unified, SWA, Mamba, HiCache, and LMCache paths reused the prefix-key idea while changing what cached state contains and where it resides. It widened a final time when registered tests, cache events, benchmark shared-prefix workloads, and metrics surfaces made a cache hit auditable outside the tree implementation. Each step preserves the same logical-prefix-to-physical-state invariant while adding a new consumer of that invariant.

Streaming sessions fit into this progression because they expose another ownership form for the prefix-cache alignment rule. A later turn can recover committed KV from a session slot, but the implementation still has to remember the protected

tree prefix, free stale allocated tail slots on page boundaries, and release the original lock when the session ends. The same radix contract applies when the reusable state is temporarily owned by a session rather than by an ordinary active request.

Prefix reuse became a contract between the scheduler, allocator, cache tree, and attention layer. The tree finds reusable state, but the rest of the runtime must decide whether that state is locked, resident, page-aligned, transferable, evictable, and safe to expose to the next forward step.

8.12. Summary

RadixAttention turns repeated token prefixes into reusable runtime state when prefix identity and physical KV ownership agree. Token sequence, namespace metadata, page geometry, and protected cache state must describe the same executable prefix; the prefix-cache alignment invariant (Invariant 8.8.1) is the check.

The core transition is match, project, and execute: a prefix hit maps cached KV slots into request state, leaves suffix work explicit, and uses publication rules to decide which new state becomes reusable. Lifetime rules protect live entries; eviction, streaming-session variants, trace rows, and hierarchical tiers keep executable reuse separate from textual overlap.

8.13. Exercises

1. A radix tree stores a path for tokens [1,2,3,9]. A new request begins with [1,2,3,9,4]. Identify the reusable prefix and the uncached suffix.
2. With page size 16, a request has a textual prefix match of 103 tokens. What page-aligned prefix length can the tree publish, and how many matched tail tokens remain outside the tree?
3. Explain why prefix identity must include cache namespace metadata rather than only token ids.
4. A cache match returns 96 physical KV indices. What does the scheduler do differently for a 128-token prompt than it would do with no hit?
5. Describe the node split when an existing edge [10,11,12,13] receives a new key [10,11,99] with page size 1.
6. Why does an eviction policy need lock references in addition to a least-recently-used or priority ordering?
7. Compare finished-request insertion with unfinished-request insertion during chunked prefill.
8. Explain how a streaming-session virtual node can produce a reusable prefix without being an ordinary evictable radix-tree node.
9. Design a page-alignment experiment that records prompt length, page size, matched/published prefix lengths, KV pages, and recomputation count. Include a same-text/different-alignment negative case and state the cache-key boundary and scheduler-visible prefill length for both.
10. For raw tokens [5,6,7,8], list the EAGLE-style bigram units, explain the key-length difference, and name one ordinary-token metadata bug plus the invariant and trace evidence that catch it.